

# 作品设计报告

课程名称: 基于软硬件协同设计的 FPGA 项目实战

作品名称: 个性化示波器

实验日期: 2026 年 5 月 10 日 地 点: K421

学 号: 2024311439 姓名: 杨嘉龙

学 号: 2025310272 姓名: 罗昊

学 号: \_\_\_\_\_ 姓名: \_\_\_\_\_

评分: \_\_\_\_\_

---

教师评语:

教师签字: \_\_\_\_\_ 日 期: \_\_\_\_\_

# 1 设计概述

## 1.1 设计目的

作为简易的示波器显示波形以及其频率、幅值等基本量

## 1.2 应用领域

当实验室没有示波器时用于暂时替代发挥示波器的功能

## 1.3 主要技术特点

### 1. 严格的多时钟域隔离设计

系统采用三时钟域完全隔离架构，从根本上解决了高速采集与高清显示的时序冲突问题：

- **ADC 采集时钟域 (40MHz)：**负责 ADC 数据采集、按键消抖、频率 / 幅值测量、自动触发与 FIFO 写操作，所有逻辑同步于外部 ADC 输出时钟，保证采样精度
- **显示控制时钟域 (148.5MHz)：**负责 VGA 扫描、波形渲染、数字显示、图片叠加与 HDMI 输出，同步于像素时钟，保证显示流畅无闪烁
- **APB3 系统时钟域 (96MHz)：**负责 RISC-V SOC 运行与 APB3 总线数据传输，同步于系统时钟，保证总线通信稳定

跨时钟域数据交互采用异步 FIFO + 异步双端口 BRAM 的标准方案：

- 波形数据与测量结果通过 afifo\_w8d2048 和 afifo\_w20d16 异步 FIFO 传输，解决了 40MHz 与 148.5MHz 时钟域的速率不匹配问题
- RISC-V 传输的图片数据通过 bram\_test 异步双端口 BRAM 缓存，实现了 96MHz 写入与 148.5MHz 读取的交互

### 2. 模块化封装与可复用设计

工程采用模块化设计原则，将复杂系统拆分为多个独立可复用的子模块：

将 "APB3 总线接收 + BRAM 缓存 + 像素生成" 整套图片显示逻辑封装为独立的 vga\_pic\_apb 模块，对外仅暴露标准化的 APB3 从机接口与 VGA 像素接口

3. 双通道同步采集与并行测量

工程支持双通道独立采集与并行测量功能：

- 同时采集两个通道的 ADC 数据 (DAM 和 DBN)
- 每个通道独立配备频率测量模块 (frequency\_measure)、幅值测量模块 (ampl\_measure) 与自动触发模块 (auto\_trigger)
- 测量结果通过独立的异步 FIFO 传输到显示域，实现了采集与显示的分开作用

4. 标准化 APB3 总线接口与 RISC-V 异构集成

工程集成了 RISC-V 软核，通过标准化 APB3 总线实现了处理器与 FPGA 逻辑的交互：

- RISC-V 作为主机，FPGA 作为从机，通过 APB3 总线实现图片数据的高速传输
- 图片数据存储在 FPGA 内部 BRAM 中，无需外部存储器，简化了硬件设计

5. 分层显示叠加架构

显示系统采用四层叠加架构，优先级从高到低依次为：

- 图片层：RISC-V 传输的自定义图片，显示在屏幕右下角
- 数字层：频率与幅值测量结果的数字显示
- 波形层：ADC 采集的模拟波形
- 背景层：背景色彩

1.4 关键性能指标

(1) 采集性能指标

| 指标      | 数值     | 依据                            |
|---------|--------|-------------------------------|
| 采样率     | 40MSPS | ADC 时钟 clk_adc 为 40MHz        |
| 垂直分辨率   | 8 位    | ADC 数据总线宽度为 8 位               |
| 通道数     | 2 通道   | 支持 DAM 和 DBN 两路输入             |
| 单通道存储深度 | 2048 点 | 波形 FIFOafifo_w8d2048 深度为 2048 |
| 触发方式    | 自动触发   | 内置 auto_trigger 模块实现边沿触发      |

(2) 测量指标

| 指标     | 数值           | 依据                                             |
|--------|--------------|------------------------------------------------|
| 频率测量范围 | 20kHz ~ 9MHz | 满足奈奎斯特采样定理，即时钟信号频率必须不小于测量频率上限的 2 倍才可以保证测量结果不失真 |
| 幅值测量范围 | 0.5V-7.5V    | ADC 硬件是 8 位数模转换器                               |

（3）显示性能指标

| 指标     | 数值             | 依据                        |
|--------|----------------|---------------------------|
| 显示分辨率  | 1920×1080@60Hz | 像素时钟 clk_pixel 为 148.5MHz |
| 颜色深度   | 24 位真彩色        | RGB 数据总线宽度为 24 位          |
| 图片显示尺寸 | 60×60 像素 (可配置) | vga_pic_apb 模块参数可修改       |

（4）系统性能指标

| 指标     | 数值     | 依据                   |
|--------|--------|----------------------|
| 系统时钟频率 | 96MHz  | APB3 总线与 RISC-V 运行时钟 |
| 总线传输速率 | 96MB/s | 96MHz×8 位            |

1.5 主要创新点

（1）在 AI 的建议下使用 mem 文件预存储字符和数字的字模代码，定义 0-9 数字和“freq”“kHz”等字符数字寄存器，然后利用 readmemh 函数将文件中的数据加载到寄存器中。因此在给频率和幅值的各位赋值时的 case 语句的代码量会减少很多，不必每个数字的字模赋值都完整地写一遍。

```
=====initial begin
// 第一步：预定义所有字模数组（0-9 + 单位 + 标签）
//=====
// 数字0-9字模（24x48）
reg [23:0] num_0 [47:0];
reg [23:0] num_1 [47:0];
reg [23:0] num_2 [47:0];
reg [23:0] num_3 [47:0];
reg [23:0] num_4 [47:0];
reg [23:0] num_5 [47:0];
reg [23:0] num_6 [47:0];
reg [23:0] num_7 [47:0];
reg [23:0] num_8 [47:0];
reg [23:0] num_9 [47:0];

// 单位和标签字模
reg [71:0] unit_khz [47:0]; // kHz: 72x48
reg [23:0] unit_v [47:0]; // V: 24x48
reg [119:0] unit_freq [47:0]; // FREQ: 120x48
reg [119:0] unit_ampl [47:0]; // AMPL: 120x48

// 预加载所有数字字模（上电只执行一次）
$readmemh("0.mem", num_0);
$readmemh("1.mem", num_1);
$readmemh("2.mem", num_2);
$readmemh("3.mem", num_3);
$readmemh("4.mem", num_4);
$readmemh("5.mem", num_5);
$readmemh("6.mem", num_6);
$readmemh("7.mem", num_7);
$readmemh("8.mem", num_8);
$readmemh("9.mem", num_9);

// 预加载单位和标签字模
$readmemh("kHz.mem", unit_khz);
$readmemh("V.mem", unit_v);
$readmemh("freq.mem", unit_freq);
$readmemh("Ampl.mem", unit_ampl);
end

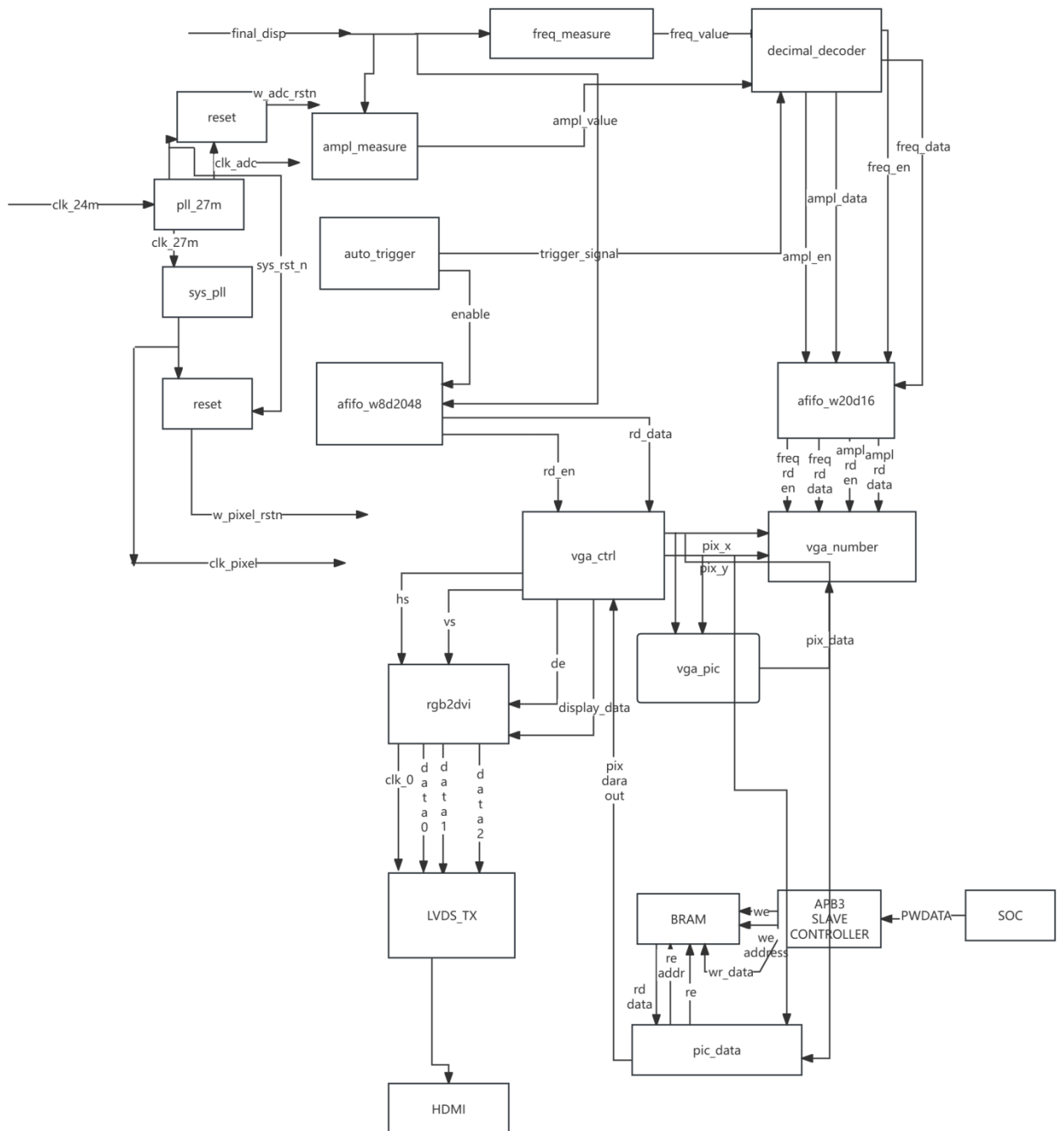
case (unit_temp)
4'd0: freq_digit0_line = num_0[char5_y];
4'd1: freq_digit0_line = num_1[char5_y];
4'd2: freq_digit0_line = num_2[char5_y];
4'd3: freq_digit0_line = num_3[char5_y];
4'd4: freq_digit0_line = num_4[char5_y];
4'd5: freq_digit0_line = num_5[char5_y];
4'd6: freq_digit0_line = num_6[char5_y];
4'd7: freq_digit0_line = num_7[char5_y];
4'd8: freq_digit0_line = num_8[char5_y];
4'd9: freq_digit0_line = num_9[char5_y];
default: freq_digit0_line = num_0[char5_y];
endcase
```

（2）在频率显示模块设置了一个“button\_state”按键状态寄存器，在其位于低电平时显示低频

测量值(单位改为 Hz,将原测得频率值乘以 1000)。但问题在于超出低频测量范围(0-99999Hz)后低频通道的 5 位数字全部变为 0.改进思路为添加提示词替代原有的显示。

## 2 系统组成及功能说明

### 2.1 整体方案设计



## 2.2 各模块设计与实现

### 1. PLL 时钟模块

**模块功能：**将开发板上单一的 24MHz 晶振，转换为系统各个模块所需的不同频率、不同相位的稳定时钟。

**设计原理：**采用两级 PLL 级联架构，分别生成 ADC 采样时钟域和 HDMI 显示时钟域的时钟信号。第一级 PLL 生成 40MHz ADC 采样时钟和 27MHz 中间时钟；第二级 PLL 将 27MHz 转换为 74.25MHz 像素时钟和 371.25MHz LVDS 串行时钟以及 96MHz 的 APB 时钟。

### 2.Reset 模块

（1）**功能：**生成高低两种复位信号

（2）**端口分析：**

|            |           |                      |
|------------|-----------|----------------------|
| input      | clk_i,    | 模块的时钟，与系统的 ADC 时钟相连接 |
| input      | locked_i, | 模块的复位，与系统的复位信号连接     |
| output reg | rstn_o,   | 低电平复位信号              |
| output reg | rst_o     | 高电平复位信号              |

**实现过程：**将 pll 输出的 lock 信号进行打拍处理。在 pll 锁定后延迟 16 个周期释放复位信号（先强制复位，再利用移位寄存器进行资源节省型的打拍处理）

## ADC 传输部分

### 1. ampl\_measure

（1）**端口**

input clk\_adc 40MHz ADC 采样时钟

input rst\_n 低电平复位

input [7:0] adc\_data 8 位 AD9288 采样数据（偏移码：0=-5V，255=+5V）

input trigger\_signal 触发脉冲，与信号上升沿同步，连接到 auto\_trigger 模块

output reg [31:0] ampl\_value 峰峰值，单位：mV（如 4000 表示 4.000V）用于传输给 decimal\_decoder 模块

output reg ampl\_valid 幅值更新有效信号，传输给下游的 decimal\_decoder 的 start 端口，启动将二进制数转换为 BCD 编码

（2）**实现思路**

1\*先进行触发信号边沿检测，两级打拍消除亚稳态

2\*核心测量逻辑：每个 adc\_data 进来后都会与前一个数据进行对比，若新的 data 更大，则留下，否则保留

前者。触发上升沿到来之际会锁存上一周期的测量结果，并锁存当前周期的 min/max 值

3\*单位换算：计算公式：**峰峰值(mV) = (max - min) \* 20000 / 255**

**20000 = 5V(ADC 满量程) × 4(硬件分压衰减) × 1000(转换为 mV)**

255 = 8 位 ADC 的满量程数值

## 2.freq\_measure（等精度测量法）

### （1）端口定义

**Input clk\_adc**

**Input rst\_n**

**Input adc\_data** 连接存储 ADC 写入数据的 r\_dataA/B，并通过检测其上升沿计算频率

**Input button\_stata** 刻画按键状态，高电平时显示高频，低电平时显示低频

**Output reg [31:0] freq\_value** 储存 32 位频率测量值，连接

**Output reg freq\_valid** 连接 decimal\_decoder（定义与幅值部分相仿）

### （2）实现思路

1\*进行参数配置：基准时钟频率 软件闸门时间（此时需要考虑平衡精度和响应速度）软件闸门计数最大值

2\*消除亚稳态并且检测过零点上升沿（设置 adc\_d1 adc\_d2）

```
// 两级打拍消除亚稳态
reg [7:0] adc_d1, adc_d2;
always @(posedge clk_adc or negedge rst_n) begin
    if(!rst_n) begin
        adc_d1 <= 8'd128;
        adc_d2 <= 8'd128;
    end else begin
        adc_d1 <= adc_data; // 第一级打拍
        adc_d2 <= adc_d1;   // 第二级打拍
    end
end
```

```
// 检测过零点上升沿（128是AD9288的0V偏移）
reg adc_pos_edge;
always @(posedge clk_adc or negedge rst_n) begin
    if(!rst_n) begin
        adc_pos_edge <= 1'b0;
    end else begin
        adc_pos_edge <= (adc_d2 < 8'd128) && (adc_d1 >= 8'd128);
    end
end
```

3\*设置软件闸门与实际闸门，测量时钟脉冲数和被测信号的脉冲数并计算频率

4\*根据按键状态判断是否应当\*1000



## 优化方案

### 1. 幅值

（1）**上限**：9288 是八位的，对应模拟输入为 0-5V，由于前端电路的四倍分压衰减，所以峰峰值测量上限为  $4 \times 5V = 20V$

（2）**下限**：最小分辨率  $1LSB = 20V / 255 = 78.4mV$ ，且噪声大小是 2-3 个 LSB，因此测量下限在 300mV 附近。

（3）**优化方案**（代码层面）

A. 增添多级滤波，可使噪声下降到 1LSB 左右

B. 添加**硬件程控增益放大器（PGA）**，先将输入的小信号经过放大再送入 ADC，可以降低 LSB 的值

### 2. 频率

（1）**上限**：约 10MHz。因为 ADC 采样率是 40MHz，**根据奈奎斯特采样定理**，理论上最高能测量 20MHz 的信号。

但为了采集有效的上升沿，每半个周期需要两个点，每个周期至少需要 4 个点，所以实际准确的采样频率上限应为 10MHz（超出后会导致信号的跳变等不正常的显示）

**硬件扩展空间**：换用更高速的 ADC，实现 100MSPS 以上的采样率

（2）**下限**：0.1Hz。因为环境噪声的干扰

**代码优化**：延长软件闸门的时间（同时会导致实际闸门的时间延长）

### 3. auto\_trigger 触发模块

(1) 功能分析: 每隔 100ms 采样 1920 个 ADC 数据点, 把 ADC 数据写入 FIFO 以供读取和显示

(2) 端口分析:

**parameter ENABLE\_COUNT = 1920** 每次采集的点数

**parameter COUNT\_MAX1s = 40\_000\_00** 采集间隔计数器最大值

**Input wire Clk\_40mhz**

**Input wire Reset**

**Input adc\_pos\_edge** 当检测到信号上升沿过 0 时, 产生单周期脉冲, 启动一次新的波形采集

**Output reg enable** 高电平时允许 ADC 数据写入 FIFO; 低电平时禁止数据写入, 连接 FIFO 核的 wr\_en 端口

**Output wire[\$clog2(ENABLE\_COUNT)-1:0] addr** 从 0-1919 循环计数, 告诉 FIFO 当前的 ADC 数据应该写到哪个地址。连接到 FIFO 核的 wr\_addr 端口

**Output reg trigger\_signal** 每次采集完成时 (100ms 后), 产生一个单周期脉冲, 告诉测量模块可以开始计算新的频率和幅值, 连接频率幅值测量模块的 start 端口

(3) 实现思路:

40MHz 时钟驱动整个模块运行, 当检测到信号上升沿时 (adc\_pos\_edge), enable 拉高开始采集, addr 从 0 开始计数, 每来一个 ADC 数据就写入 FIFO 对应的地址; 采集满 1920 个点后, enable 拉低停止采集, 同时 trigger\_signal 输出一个脉冲, 启动频率和幅值测量, 等待下一个上升沿到来重复上述过程。

#### 4. decimal\_decoder 模块

(1) **功能分析：**将输入的 32 位二进制数据通过逐位除 10 取余赋值给频率、幅值的各个数位

(2) **端口分析：**

```
input  wire      clk,           // 时钟信号
input  wire      reset,        // 复位信号
input  wire [31:0] binary_input, // 输入的 32 位二进制数
input  wire      start,        // 开始信号,接收来自 auto_trigger 模块的 trigger_signalA(B)信号,
```

启动进制转化模块的运行

```
output reg [3:0] units,         // 个位
output reg [3:0] tens,         // 十位
output reg [3:0] hundreds,     // 百位
output reg [3:0] thousands,    // 千位
output reg [3:0] ten_thousands, // 万位
output reg      done           // 完成信号,用于传输给下游的 FIFO 作为写使能信号,启动频
```

率幅值测量值（转为十进制后）写入 FIFO 核

(3) **实现思路**

使用状态机写法，process 表示正在进行数制转化的状态，5 位转化完毕后进入 done 状态，再过一个上升沿后进入 IDLE 状态。在 process 状态时 temp 变量负责存储除 10 后的数据，digit\_index 存储正在处理的数位的序号（比如处理到百位了就是 3，十位是 2），每次除 10 后自增 1.直到 digit\_index 达到 5 后进入 done 状态，同时 done 信号置为 1，提示下游的 FIFO 模块可以写入数据了。

## FIFO 跨时钟预处理

### 一、波形数据存储 FIFO (afifo\_w8d2048)

#### 1. 模块参数

数据位宽：8bit；存储深度：2048；工程中实例化 2 个，分别对应示波器 A、B 通道。

#### 2. 核心功能

该 FIFO 为 **ADC 原始波形数据专用缓存单元**，用于暂存 ADC 采集的实时采样数据，完成跨时钟域数据传输，为 VGA 控制器提供稳定的波形绘制数据源。

#### 3. 工作机制

**数据写入：**工作于 clk\_adc 时钟域，由自动触发模块 auto\_trigger 输出写使能信号，触发后连续写入 1920 个 ADC 采样点，完成一帧波形数据的存储；

**数据读取：**工作于 clk\_pixel 时钟域，读使能信号由外部 VGA 扫描时序逻辑生成，当 FIFO 内存储满 1920 个数据时启动读取操作，逐点输出波形数据；

**跨时钟域隔离：**独立的读写时钟架构，实现 ADC 采样域与 VGA 显示域的数据安全交互。

#### 4. 工程应用

为 vga\_ctrl 模块提供 A、B 通道的原始波形数据，支撑屏幕波形绘制功能。

### 二、测量数值存储 FIFO (afifo\_w20d16)

#### 1. 模块参数

数据位宽：20bit（兼容 5 位 BCD 编码，每 4bit 存储一位十进制数）；存储深度：16；工程中实例化 4 个，分别对应 A 通道频率、A 通道幅值、B 通道频率、B 通道幅值的测量结果。

#### 2. 核心功能

该 FIFO 为 **频率 / 幅值测量结果专用缓存单元**，用于暂存十进制解码后的测量数值，完成跨时钟域数据传输，为 VGA 数字显示模块提供稳定的数值数据源。

#### 3. 工作机制

**数据写入：**工作于 clk\_adc 时钟域，由十进制解码模块 decimal\_decoder\_seq 的完成信号作为写使能，解码完成后写入一组 5 位 BCD 测量数值；

**数据读取：**工作于 clk\_pixel 时钟域，读使能信号由 FIFO 空标志位取反生成，FIFO 非空时自动读取数值数据；

**数据保持：**小深度存储设计满足单次测量数值的缓存需求，保证屏幕显示数值的实时性与稳定性。

#### 4. 工程应用

为 vga\_number 模块提供频率、幅值的十进制编码数据，支撑屏幕测量数值显示功能。

## VGA 显示部分

### 1. vga\_number 数字显示模块

(1) **功能分析：**接受来自 vga\_ctrl 模块的坐标信号以及频率幅值的数据和使能信号，在屏幕的相应位置通过点亮像素点实现频率与幅值数据的自定义色彩显示

(2) **端口分析：**

input wire vga\_clk 连接到 25MHz 系统采样时钟

input wire sys\_rst\_n 系统低电平复位信号

input wire [11:0] pix\_x

input wire [11:0] pix\_y 接收来自 vga\_ctrl 模块的坐标信息

// 频率数据接口

input wire freq\_freq\_rd\_en,

input wire [19:0] freq\_rd\_data,

// 幅值数据接口

input wire ampl\_ampl\_rd\_en,

input wire [19:0] ampl\_rd\_data,

Input button\_state 按键切换信号，高电平时负责低频，反之负责高频

// 像素输出

output reg [23:0] pix\_data 在指定区域被赋予自定义色彩，传输给 vga\_ctrl 模块并被赋值给 rgb 信号。

(3) **实现思路**

1\*把外部字模文件加载到数字 0-9、字符数组中

2\*接收到来自频率、幅值测量模块的使能信号后锁存来自 FIFO 核的频率、幅值测量数值

3\*为进行分屏显示。设置两个基线起始位置。并定义字符的宽高与起始位置

4\*计算五个数位、字符的显示区域，即所谓“字符空间内的相对坐标”，后续显示逻辑用得上

5\*先定义位寄存器，用 case 语句对频率和幅值的各数位赋值。同时根据预设的分区显示在左右特定区域实现频率、赋值字符 (freq ampl) 及其数值的显示。Button\_state 低电平时允许 kHz 显示，需要在三目运算符中多一行关于它的判断

## 2.vga\_ctrl 显示控制模块

(1) **功能分析:** 作为显示核心控制中枢, 负责生成标准 1080P@60Hz 的 HDMI 时序, 同时集成了双通道波形的缓存、坐标转换和显示逻辑, 是连接所有显示子模块(字符、数字、波形、Logo)和最终 HDMI 输出的桥梁。

(2) **端口分析:**

|        |      |         |                    |                                             |
|--------|------|---------|--------------------|---------------------------------------------|
| input  | wire |         | vga_clk            | 输入工作时钟,频率 25MHz                             |
| input  | wire |         | sys_rst_n          | 输入复位信号,低电平有效                                |
| input  | wire | [23:0]  | pix_data           | 接受自 vga_number vga_pic 的像素点信息               |
|        |      |         |                    | FIFO 读使能信号, 高电平时允许从储存 ADC 数据的 FIFO 核中读取波形数据 |
|        |      |         |                    | 从 FIFO 核读出的 8 位 ADC 数据                      |
| input  |      |         | bram_en_A          |                                             |
| input  |      | [ 7:0]  | bram_data_A        |                                             |
| input  |      |         | bram_en_B          |                                             |
| input  |      | [ 7:0]  | bram_data_B        |                                             |
|        |      |         |                    | 输出波形最终像素 (A 通道 B 通道)                        |
| output |      | [ 23:0] | wave_displaydata_A |                                             |
| output |      | [ 23:0] | wave_displaydata_B |                                             |
| output | wire | [11:0]  | pix_x              | 输出 VGA 有效显示区域像素点 X 轴坐标                      |
| output | wire | [11:0]  | pix_y              | 输出 VGA 有效显示区域像素点 Y 轴坐标                      |
|        |      |         |                    | 三者需要作为输入信号传输给 rgbdvi2 模块                    |
| output | wire |         | hsync              | 输出行同步信号                                     |
| output | wire |         | vsync              | 输出场同步信号                                     |
| output | wire |         | rgb_valid          | VGA 有效显示区域                                  |
| output | wire | [23:0]  | rgb                | 输出像素点色彩信息                                   |

(3) **实现思路**

1. 定义标准时序参数
2. 定义行场计数器, 行计数器与时钟周期相同, 数到 (2200-1) 后清零, 表示一行扫描完成; 场计数器在行计数器完成一整行计数后才+1.1s 内可以完成一幅屏幕的 60 次扫描

3. 标记有效区域,并将两个时序计数器经过坐标转换后分别变成 1920、1080 的范围,然后赋值给 `pix_x,pix_y`
4. 定义波形缓存数组,并在 FIFO 读使能信号置高时,将 FIFO 中的采样点写入缓存数组。
5. 由于 HDMI 显示的纵坐标是自上而下增大,与波形显示要求的自下而上增大相反,必须进行一步波形翻转。
6. 显示“行号 `ypos_bram_A` 正好等于该列的 ADC 采样 `wave_temp_A[xpos_bram]`”的像素点为白色,其余为黑色。

### 3.vga\_pic 示波器顶部文字显示模块

(1) 功能分析: 在接收到 `vga_ctrl` 模块的有效横纵坐标信号后,先导入“规格严格功夫到家”的字模代码,再于目标显示区域内根据字模代码点亮或熄灭确定坐标处的像素

(2) 端口分析:

```
input  wire      vga_clk      , //输入工作时钟,频率 25MHz
input  wire      sys_rst_n    , //输入复位信号,低电平有效
input  wire [11:0] pix_x      , //输入有效显示区域像素点 X 轴坐标
input  wire [11:0] pix_y      , //输入有效显示区域像素点 Y 轴坐标
```

```
output reg [23:0] pix_data      //输出像素点色彩信息给 vga_ctrl 模块,这部分数据最终由
vga_ctrl 传输给 rgb2dvi,于此将并行视频信号编码转化为 tmds 差分串行信号(10 位)
```

(3) 实现方式:

先定义字符位置、宽高、颜色等基本参数,定义字符显示位置的坐标和字模代码的寄存器,并导入存入字模数据的 `mem` 文件。最后在指定区域随着行场同步信号对寄存器逐行逐列地赋值、点亮

//pix\_data:输出像素点色彩信息,根据当前像素点坐标指定当前像素点颜色数据

```
always@(posedge vga_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        pix_data <= BLACK;
    else if((((pix_x >= (CHAR_B_H - 1'b1))
        && (pix_x < (CHAR_B_H + CHAR_W - 1'b1)))
        && ((pix_y >= CHAR_B_V) && (pix_y < (CHAR_B_V + CHAR_H))))
        && (char[char_y][511 - char_x] == 1'b1))
        pix_data <= DIY_COLOR;
```

基于 RISCv APB3 的图片显示部分

1. pic\_data\_generator 图片像素生成模块

(1) 功能分析

bram 核中接收来自 APB3 总线的图像数据，当扫描到图像显示的区域（右下角 60\*60），读使能信号被拉高，通过一级打拍后赋值给图片有效信号 pic\_valid。当其被置高时，bram 核中的 rd\_data 数据被传输给像素输出数组 pix\_data\_out（取低 24 位作为 rgb 输出）。其他区域就被显示为背景色。

(2) 端口分析

| 端口类型 | 端口名          | 位宽    | 功能说明                       |
|------|--------------|-------|----------------------------|
| 输入   | vga_clk      | 1bit  | VGA 像素时钟（1080P 为 148.5MHz） |
| 输入   | sys_rst_n    | 1bit  | 系统低电平复位信号                  |
| 输入   | pix_x        | 12bit | 当前扫描像素 X 轴坐标               |
| 输入   | pix_y        | 12bit | 当前扫描像素 Y 轴坐标               |
| 输入   | rd_data      | 32bit | 双端口 BRAM 读出的图像数据           |
| 输出   | rd_en        | 1bit  | BRAM 读使能信号，高电平有效           |
| 输出   | rd_addr      | 14bit | BRAM 读地址信号                 |
| 输出   | pix_data_out | 24bit | VGA 最终输出像素数据（RGB888）       |
| 参数   | H_VALID      | 11bit | 行有效像素数（默认 1920，1080P）      |
| 参数   | V_VALID      | 11bit | 场有效像素数（默认 1080，1080P）      |
| 参数   | H_PIC/W_PIC  | 10bit | 显示图片的宽 / 高（默认 60×60）       |
| 参数   | PIC_SIZE     | 14bit | 图片总像素数（默认 3600）            |

(3) 实现思路

- 1\*在读使能信号有效期间，读地址自增 4，直到地址参数后清零
- 2\*设定读使能信号有效的区域（本程序指定在右下角）
- 3\*将读使能信号打拍后传递给图片有效信号 pic\_valid（对齐 bram 读延迟）最后将 bram 数据输出给 pix\_data\_out。



2. APB3\_SLAVE\_CONTROLLER 从机模块

（1）**功能分析：**负责将处理器通过 APB3 总线发送的 8 位像素数据，拼接为 24 位标准 RGB 格式并写入双端口 BRAM，为 VGA 显示系统提供可动态更新的图片数据源。

（2）**端口分析：**

| 端口名       | 方向 | 位宽         | 功能描述                                               |
|-----------|----|------------|----------------------------------------------------|
| sys_clk   | 输入 | 1          | 模块工作时钟，连接系统 PLL 输出的 96MHz 时钟，与 RISC-V 核、BRAM 写时钟同步 |
| sys_rst_n | 输入 | 1          | 低电平有效复位信号，连接系统复位模块输出，复位时清空所有寄存器与计数器                |
| PADDR     | 输入 | 16         | APB3 地址总线，本设计为 16 位，用于寻址 BRAM 的像素存储地址              |
| PSEL      | 输入 | 1          | 从机选择信号，高电平时表示处理器选中本模块进行数据传输                        |
| PENABLE   | 输入 | 1          | 总线使能信号，高电平时表示进入 APB3 总线访问阶段                        |
| PWRITE    | 输入 | 1          | 读写控制信号，高电平为写操作，低电平为读操作                             |
| PWDATA    | 输入 | DATA_WIDTH | APB3 写数据总线，本设计为 8 位，传输处理器发送的单字节像素分量数据              |
| PREADY    | 输出 | 1          | 从机就绪信号，高电平时表示从机已准备好接收 / 发送数据，本设计为零等待响应             |
| PRDATA    | 输出 | DATA_WIDTH | APB3 读数据总线，本设计未实现读功能，固定输出 0                        |
| PRDATA_EN | 输出 | 1          | 读数据有效信号，本设计未实现读功能，仅做协议兼容                           |
| PSLVERROR | 输出 | 1          | 总线错误响应信号，本设计不支持错误响应，固定输出 0                         |
| we        | 输出 | 1          | BRAM 写使能信号，高电平时允许将数据写入 BRAM                        |

| 端口名   | 方向 | 位宽 | 功能描述                                            |
|-------|----|----|-------------------------------------------------|
| waddr | 输出 | 14 | BRAM 写地址总线，寻址范围 0~14399，对应 60×60 图片的 3600 个像素地址 |
| wdata | 输出 | 32 | BRAM 写数据总线，高 8 位补 0，低 24 位为拼接完成的 RGB 像素数据       |

**（3）实现思路：**模块整体分为 **APB3 总线协议解析**、**RGB 像素数据拼接**、**BRAM 写控制**三个核心部分，采用同步时序设计，所有逻辑均在 96MHz 系统时钟下运行。

**1. APB3 总线协议实现（核心状态机）**

采用三段式状态机实现标准 APB3 协议，定义三个状态：

- **IDLE（空闲态）：**总线无操作时的默认状态，当检测到 PSEL=1 且 PENABLE=0 时，跳转至 SETUP 状态。
- **SETUP（建立态）：**总线地址与控制信号建立阶段，当检测到 PENABLE=1 时，跳转至 ACCESS 状态。
- **ACCESS（访问态）：**总线数据传输阶段，本设计为零等待响应，PREADY 直接拉高，完成一次数据传输后跳转回 IDLE 状态。

通过 actWrite 与 actRead 信号标记当前总线操作类型，仅响应写操作，读操作返回固定值 0。

**2. RGB 像素数据拼接逻辑**

由于 APB3 总线数据位宽为 8 位，而标准 RGB 像素为 24 位（R/G/B 各 8 位），因此需要将 3 次连续的 8 位写操作拼接为 1 个 24 位像素：

1. 定义 4 位计数器 rgb\_cnt，计数范围 0~2，记录当前已接收的字节数；
2. 每次 APB3 写事务完成时（pi\_flag=1），计数器加 1；
3. 当计数器计到 2 时，表示已接收完整的 R/G/B 三个字节，生成 wr\_rgb\_en 脉冲信号；
4. 采用左移拼接方式：wr\_rgb\_data = {PWDATA, wr\_rgb\_data[23:8]}，新数据写入高 8 位，旧数据右移 8 位，与软件发送的 B→G→R 字节顺序匹配；
5. 最终输出 32 位写数据：高 8 位补 0，低 24 位为拼接完成的 RGB 数据。

**3. BRAM 写地址与使能生成**

1. **地址计算：**定义参数 ADDR\_RGB\_END = PIC\_SIZE \* 4 - 4，表示图片最后一个像素的 BRAM 地址（每个像素占 4 字节，与 32 位 BRAM 位宽匹配）；
2. **地址递增：**每次 wr\_rgb\_en 有效时，写地址 waddr 加 4，保证每个像素占用连续的 4 字节地址空间；
3. **循环写入：**当写地址达到 ADDR\_RGB\_END 时，自动清零地址，实现图片的循环覆盖写入；
4. **写使能控制：**BRAM 写使能 we 直接连接 wr\_rgb\_en，仅在像素拼接完成时允许写入 BRAM。



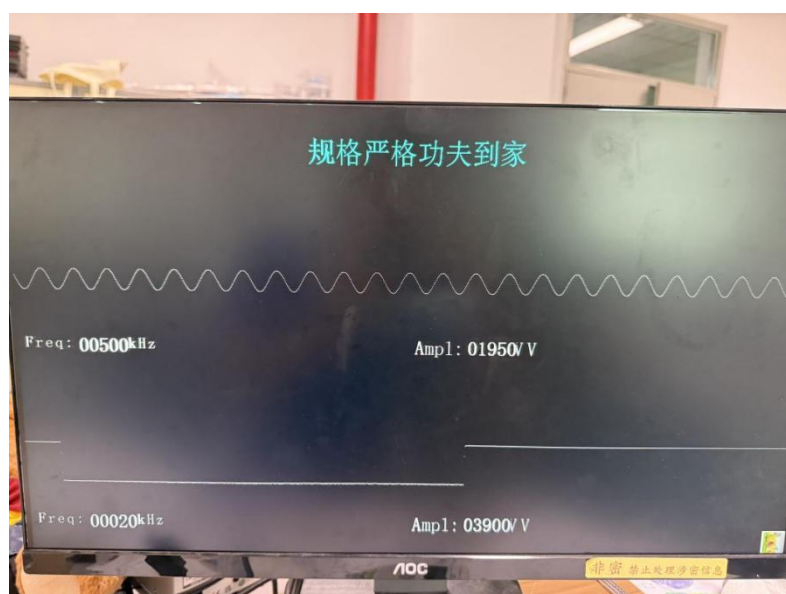
### 3 完成情况

#### 显示和误差

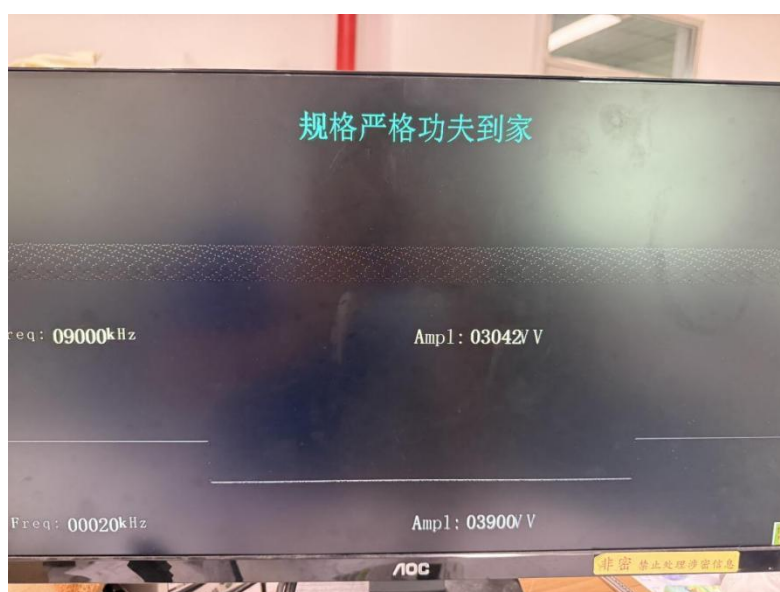
(1) 频率输入位于 $<20\text{kHz}$ 这一区间内时受量程限制会显示为 $20\text{kHz}$ , 高于 $20\text{kHz}$ 后能稳定显示正确的频率, 伴随着频率的改变幅值的显示比较稳定。

幅值的测量下限可到 $100\text{mV}$ , 但幅值变小后会导致频率显示的严重失真, 抖动非常剧烈。

在输入频率为 $100\text{—}900\text{kHz}$ , 输入 $V_{PP}$ 为 $4\text{V}+$ 时, 两者显示都是稳定和准确的。但 $V_{PP}$ 超出 $15\text{V}$ , 频率来到 $\text{MHz}$ 级别时, 两者的显示又变得极其不稳定。



$500\text{kHz}$   $4\text{V}(V_{PP})$ 以及 $20\text{kHz}$   $8\text{V}(V_{PP})$ 能稳定显示, 但幅值的误差比较大



频率测量上限是 $9\text{MHz}$ , 但此时幅值显示已经完全失真 (输入的 $V_{PP}$ 是 $9\text{V}$ , 理论上应该显示 $4.5\text{V}$ )



在低频显示下，双通道均能以比较高的准确度显示测量数据

(2) 频率和幅值的稳定性似乎呈现相关关系。当 VPP 增大到 8+V 时，频率的抖动基本消除，而且精准度也有了提高；将频率增大到 100kHz 以上时，幅值的显示也相对更稳定和精确。

## 显示创新点

(3) 可以通过按键切换高低频的显示。下图是 20kHz 转变为 20000Hz 显示

## 4 总结

### 4.1 作品可扩展之处

- (1) 可以基于不同波形的不同特征进行波形的判断
- (2) 通过按键实现波形的固定、放大和光标测量功能
- (3) 通过优化程序和硬件实现更大范围和更高精度的测量

## 4.2 心得体会及建议

队员 1:

杨嘉龙：在本次项目排错与调试工作中，我收获了十分实用的问题排查思路。以往排查程序漏洞时，我常会主观臆断，优先怀疑结构复杂、编写难度大的模块，盲目进行修改调试，不仅耗费大量时间精力，还难以精准定位问题根源。经过本次实践我深刻明白，高效排查故障不能凭主观经验预判易错模块，而是要严格依照程序整体运行流程，先筛选出已经调试完成、功能稳定、经过实际验证无误的程序部分，以此作为可靠基准，再顺着数据流向、功能执行顺序，逐个对剩余功能模块进行单独测试、逻辑核验与思路验证，循序渐进缩小错误范围，精准锁定故障点位。这种循序渐进、依托流程分步验证的排错方式，条理清晰且逻辑性强，能够大幅提升调试效率，也让我对模块化开发、程序整体运行逻辑有了更加全面透彻的理解。

队员 2:

罗昊：选这门课程给我带来很大收获和成长，它让我坚持不懈地 debug，培养我执行工程时的框架意识和模块意识，磨砺我遇到困难沉下心来解决的强大意志，让我养成了没事多跑 K420 的习惯。过程中我确实因为自身知识水平和课程难度，遇到了很大的问题，但我通过一遍遍阅读指导书、请教老师与学长，逐步攻克了一个个难题。即便最终结果并不十分令人满意，而且在幅值、频率的测量范围的形成原因和扩展方法上都没有太好的思路，但我仍然认为我在大一下为课程投入的时间精力是值得的。

## 5 附录

### 5.1 分工说明

杨嘉龙：本次项目开发过程中，我主要承担示波器双通道波形显示功能与按键模式切换逻辑的设计、编写及调试工作，完成双通道数据采集同步、波形渲染展示、按键触发切换显示通道等核心功能；同时协助团队成员共同完成 APB3 总线数据显示模块的联合调试，配合梳理数据传输链路，协助排查通信交互、数据读取与界面展示过程中出现的各类程序报错与逻辑异常问题，合力推进整体功能正常运行。

罗昊：基于杨嘉龙已实现的双通道、频率测量及显示的成果，实现 1.通过按键切换不同通道的

波形显示和频率幅值数据的显示（原先是一起显示的，但最终还是采用按键切换高低频显示的做法） 2.上方文字修改为“规格严格功夫到家” 3.初步整合两个例程并编写了新的顶层文件 4.利用 mem 文件存储和导入字模数据，简化了代码量 5.完成设计报告与 PPT